

สรุปภาพรวมเนื้อหาบทเรียนสัปดาห์ที่ 2 (Week 2 Snapshot) ในเวลา 5 นาที สำหรับการบันทึกการเรียนรู้ (Learning Logging) โดยยึดหลักการเขียนโค้ดอย่างมีจริยธรรม โปร่งใส และรับผิดชอบ (AI Ethics, XAI, Responsible AI) ตามที่กำหนดไว้ในเป้าหมายของรายวิชาครับ

🕒 สรุปเนื้อหา Week 2: ระบบควบคุมทิศทางโปรแกรม และการประมวลผลข้อมูลอย่างมีจริยธรรม (5-Min Snapshot)

ในสัปดาห์ที่ 2 นี้ เราได้เรียนรู้หัวใจสำคัญของการทำให้คอมพิวเตอร์ "คิดและตัดสินใจ" ได้ ผ่านโครงสร้างเงื่อนไข (Conditionals) รวมถึงสเปกตรัมของการเขียนโค้ดให้สั้นกระชับด้วย List Comprehension ซึ่งในมุมมองของ ****Responsible AI**** นั้น ทุกๆ เงื่อนไขที่เราเขียนคือการสร้าง "กฎเกณฑ์ที่ส่งผลกระทบต่อผู้ใช้งาน"

เนื้อหาถูกแบ่งออกเป็น 3 หัวข้อหลัก ดังนี้ครับ:

1. ตัวดำเนินการเปรียบเทียบและตรรกศาสตร์ (Relational & Logical Operators)

* **Technical Core:** การใช้เครื่องหมายเปรียบเทียบ (`'=='`, `'!='`, `'<'`, `'>'`, `'<='`, `'>='`) เพื่อคืนค่าเป็นจริงหรือเท็จ (Boolean: `'True'` / `'False'`) และใช้ตัวดำเนินการตรรกศาสตร์ (`'and'`, `'or'`, `'not'`) มาเชื่อมโยงเงื่อนไขเข้าด้วยกัน

* **Responsible AI Connection (Fairness):** การใช้คำสั่ง `'and'` หรือ `'or'` ต้องคำนึงถึง "ความครอบคลุมและเป็นธรรม" ตรวจสอบให้แน่ใจว่าเงื่อนไขที่เราตั้งขึ้นไม่ได้ไปตัดสินสิทธิ์หรือสร้างความลำเอียง (Bias) ให้กับข้อมูลกลุ่มใดกลุ่มหนึ่งโดยไม่เจตนา

2. การควบคุมทิศทางแบบแยกกิ่งก้าน (if-elif-else & Nested Conditionals)

* **Technical Core:** การใช้ `'if-elif-else'` เพื่อสร้างโครงสร้างการตัดสินใจ และการซ้อนเงื่อนไขภายใน (Nested if) เพื่อคัดกรองข้อมูลอย่างละเอียด เช่น ระบบคำนวณเกรด หรือระบบแนะนำเนื้อหาภาพยนตร์ตามช่วงอายุ

* **Explainable AI (XAI) Connection:** โครงสร้างเงื่อนไขเป็นตัวอย่างที่ดีที่สุดของ "White-Box Model" (โมเดลที่โปร่งใสอธิบายได้) การแยกแยะกรณีต่างๆ อย่างชัดเจนและมีการเขียนอธิบายกำกับ จะช่วยให้ผู้อื่นสามารถตรวจสอบย้อนกลับได้ว่าระบบใช้เกณฑ์อะไรในการคัดกรองหรือตัดสินใจระบบ

```
```python
```

```
=====
LEARNING LOG: ตัวอย่างระบบตัดสินใจที่โปร่งใสและตรวจสอบได้ (Explainable AI Tree)
=====
```

```
จำลองระบบการแนะนำการจัดระดับเนื้อหา (Movie Rating Matrix) ตามช่วงอายุ
```

```
try:
```

```
 age = int(input("กรุณาระบุอายุของผู้ใช้งานเพื่อแนะนำเนื้อหา: "))
```

```
โครงสร้างการตัดสินใจที่โปร่งใสและตรวจสอบย้อนหลังได้ (Transparent Criteria)
```

```

if age < 5:
 recommendation = "การ์ตูนสำหรับเด็กปฐมวัย"
elif 5 <= age <= 12:
 recommendation = "ภาพยนตร์ทั่วไป (เรท G/PG)"
elif 13 <= age <= 17:
 recommendation = "ภาพยนตร์สำหรับวัยรุ่น (เรท PG-13)"
else:
 recommendation = "เนื้อหาทั่วไปไม่จำกัดการเข้าถึง (Unrestricted Access)"

print(f"[XAI Output] ระบบประมวลผลสำหรับอายุ {age} ปี -> แนะนำ: {recommendation}")

except ValueError:
 print("[Safety Alert] ข้อมูลนำเข้าไม่ถูกต้อง! ระบบปฏิเสธการประมวลผลเพื่อความปลอดภัย")

...

3. การย่อรูปโครงสร้างข้อมูล (List Comprehension)

* **Technical Core:** การสร้าง List รูปแบบใหม่ที่กระชับและทำงานได้รวดเร็วขึ้นในบรรทัดเดียว มี
โครงสร้างคือ `[ expression for item in iterable if condition ]`
* **AI Ethics & Scannability Connection:** แม้ว่า List Comprehension จะช่วยให้โค้ดทำงานเร็วขึ้น
และสั้นลง แต่หากใส่เงื่อนไขที่ซับซ้อนเกินไปซ้อนกันหลายชั้น จะทำให้โค้ด "ไม่อ่านยาก" ซึ่งขัดกับหลัก
XAI ที่เน้นความโปร่งใส ดังนั้นหากโค้ดเริ่มซับซ้อน ควรกลับไปใช้โครงสร้าง `for loop` แบบปกติเพื่อความ
โปร่งใสในการตรวจสอบ (Readability over Cleverness)

```python
# =====
# LEARNING LOG: การประมวลผลข้อมูลอย่างมีประสิทธิภาพและเปิดเผย (Code Scannability)
# =====

# ตัวอย่างการคัดกรองและประมวลผลตัวเลขโดยใช้ List Comprehension
# โจทย์: ตรวจสอบและยกกำลังสองเฉพาะตัวเลขที่เป็นบวกและมีค่าน้อยกว่า 10
numbers = [-3, 2, 5, 11, 0, 7, -1]

# เขียนอย่างกระชับแต่คงไว้ซึ่งความชัดเจนของเงื่อนไข (Explicit Logic)
approved_squares = [x**2 for x in numbers if x >= 0 and x < 10]

print(f"ข้อมูลดิบนำเข้า: {numbers}")
print(f"ผลลัพธ์หลังการคัดกรองอย่างโปร่งใส: {approved_squares}")

...

---
```

💡 บันทึกการเรียนรู้เพื่อการพัฒนาอย่างรับผิดชอบ (Responsible Developer Prompts)

สรุปจากกิจกรรมกลุ่ม ****Choose Your Own Adventure**** และ Worksheet ในสัปดาห์นี้:

1. ****Edge Cases (การจัดการค่าวิกฤต):**** ในสัปดาห์นี้เราเน้นย้ำเรื่องการทดสอบค่ารอยต่อ (เช่น ข้อมูลที่เป็นเลข 0, ค่าติดลบ หรือข้อมูลว่างเปล่า) โค้ดที่ดีต้องไม่พังเมื่อเจอข้อผิดพลาดเหล่านั้น
2. ****Trade-off (ความสมดุล):**** ระหว่างการเขียนโค้ดให้สั้นที่สุด (List Comprehension ชับซ้อน) กับการเขียนให้คนอื่นเข้าใจง่ายที่สุด (Simple For-Loop) เราต้องเลือกสิ่งที่มีความโปร่งใสและอธิบายได้ (Explainable) มากที่สุดเป็นหลักในการทำงานร่วมกับผู้อื่นเสมอ

(หมายเหตุ: Class Activities สัปดาห์นี้มีกำหนดส่งรายงานผลการศึกษาในวันที่ 10 กรกฎาคม 2026 อย่าลืมนำแนวคิดการดักจับข้อผิดพลาดและโครงสร้างการตัดสินใจไปใส่ในเล่มรายงานด้วยนะครับ!)